VITyarthi Practical Record File

Computer Vision (CSE3010)

Submitted By:

Name: Shubham Chaudhari

Reg. no.: 23BAI10412

Table of Contents

1. **Abstract** 1
2. **Introduction** 2
3. **Problem Statement** 3
4. **Methodology and Technical Approach** 4
   4.1 Data Extraction Pipeline 4
   4.2 Feature Normalization 5
   4.3 Machine Learning Model 5
5. **Results and Evaluation** 7
6. **Conclusion** 7
7. **Future Work** 7

Real-Time American Sign Language (ASL) Translation System

1. Abstract

The project develops a real-time sign language translation system which interprets finger-spelled hand gestures through American Sign Language (ASL) translation into written text. The system uses MediaPipe technology to detect hand landmarks while it employs a Random Forest Classifier to classify hand gestures. The system uses computer vision technology together with machine learning to provide users with a cost-effective solution that works through standard webcam equipment. The model shows a 79.72% accuracy rate when tested on a skeleton-based dataset which proves its ability to deliver dependable results even with limited data resources. The research develops a unique preprocessing and normalization system which connects two different types of data: abstract skeleton-based training data and actual human input. The system uses spatial normalization together with feature consistency techniques to maintain functionality when users show different hand styles through various hand positions and movements which leads to better prediction results during real-time use.

2. Introduction

The deaf community and hearing community face ongoing difficulties which hinder their ability to communicate effectively during daily life, educational activities and work situations. People who use sign language experience communication problems with non-sign language speakers because they cannot understand sign language which creates barriers to their ability to access public spaces and socialize with others. The presence of interpreters enables people to communicate better between different languages but their availability is limited and their services cannot be used during informal situations. The demand for automatic systems which convert sign language into spoken or written language has increased because these systems provide better ways for people to communicate with one another.

The project seeks to develop a real-time American Sign Language (ASL) translation system through the use of accessible tools and efficient algorithms to solve this particular problem. The main goal of this project is to develop a system which will be able to identify hand gestures and transform them into written text with very short processing time. The system will use common hardware components which include webcams to achieve a practical design that can be implemented in real-world situations. The project focuses on recognizing finger spelling from A to Z since it serves as the basic element of ASL communication which people use to spell names and technical terms and non-specific signs.

The project uses classical machine learning methods as its main approach instead of using traditional methods that depend on large image datasets and deep learning models that need extensive computational resources. Deep neural networks demonstrate exceptional abilities to recognize gestures yet they demand substantial computational power together with extensive labeled data and prolonged training periods. The project uses a Random Forest Classifier because it delivers precise results while maintaining efficient performance and understandable results. The system design enables operation in environments with restricted processing capacity while it supports quick system creation and implementation.

The system depends on strong feature engineering methods as its main component. The system extracts important hand landmarks from raw image

data through MediaPipe, which offers a structured representation of hand geometry. The team developed custom methods to process these landmarks, which maintain consistent results across various user groups and different environmental settings. The approach improves model generalization capabilities while reducing its response to hand size and positioning and hand orientation changes.

3. Problem Statement

The primary challenge encountered during this project was the nature of the available dataset. Instead of standard RGB hand images, the dataset consisted of skeletonized representations of hands.

Challenges include:

- Tracking incompatibility with MediaPipe on skeleton images
- Unordered data points leading to inconsistent feature vectors
- Domain gap between training data and real-time webcam input

4. Methodology and Technical Approach

The overall system was designed as a multi-stage pipeline consisting of data extraction, feature normalization, and machine learning-based classification. Each stage was carefully engineered to address the challenges posed by the skeleton-based dataset and to ensure consistency between training and real-time inference.

4.1 Data Extraction Pipeline

A custom data extraction pipeline was developed using OpenCV to convert raw skeleton images into structured numerical feature vectors suitable for machine learning.

Image Segmentation

The first step involved isolating the hand skeleton from the background. Since the dataset consisted of high-contrast images (dark skeleton on a white background), grayscale thresholding was applied to convert the image into a binary format. This process eliminated irrelevant background information and retained only the essential geometric structure of the hand.

Point Localization

Once the binary image was obtained, contour detection techniques were used to identify connected components corresponding to the hand structure. Using contour moments and pixel analysis, the (x, y) coordinates of significant joint points were extracted. These points represent the key locations of fingers and joints in the skeleton.

Radial Sorting

A major challenge was the absence of a fixed ordering of points in the dataset. To address this, a radial sorting algorithm was implemented. First, the centroid of all detected points was computed. Then, each point was assigned a polar angle relative to this centroid. Points were sorted based on this angle in a consistent clockwise or counterclockwise order.

This ensured that every sample produced a fixed-length feature vector of **42 elements (21 points × 2 coordinates)**, which is essential for training a machine learning model.

4.2 Feature Normalization

To make the model robust to variations in hand position, scale, and user differences, normalization techniques were applied to the extracted features.

Translation Invariance

All coordinate points were re-centered relative to the wrist (landmark 0), which was treated as the origin. This ensured that the model focused on the relative structure of the hand rather than its absolute position in the image frame.

Scale Invariance

To handle variations in hand size and distance from the camera, all coordinates were normalized by dividing them by the maximum distance between any point and the wrist. This scaling transformed the feature values into a consistent range, typically between 0 and 1.

These normalization steps significantly improved the generalization capability of the model across different users and environments.

4.3 Machine Learning Model

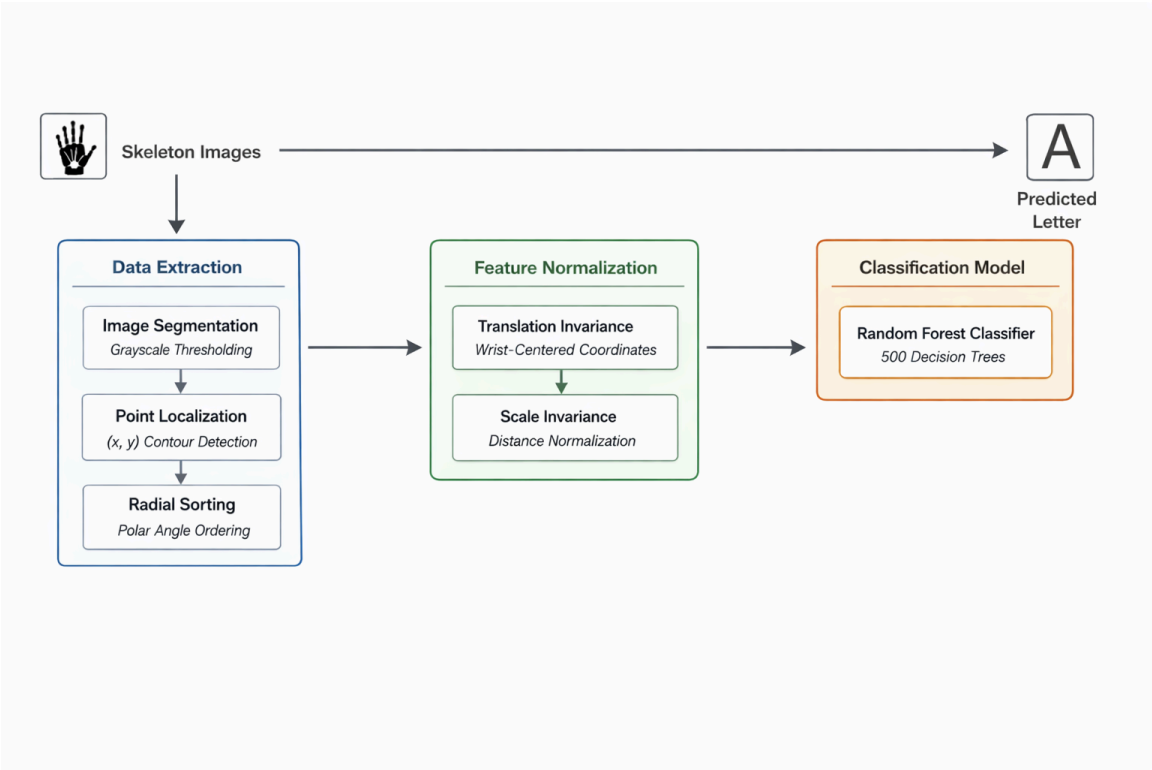
For gesture classification, a Random Forest Classifier was selected due to its effectiveness in handling structured data and its robustness to noise.

The model was configured with **500 decision trees (estimators)**, allowing it to learn complex decision boundaries while reducing the risk of overfitting through ensemble averaging.

Key advantages of using Random Forest include:

- **Robustness to Noise:** The ensemble nature helps in minimizing the impact of noisy or imperfect data points.
- **Generalization Capability:** By averaging multiple decision trees, the model avoids overfitting and performs well on unseen data.
- **Efficiency:** Compared to deep learning models, Random Forest requires less computational power and training time.
- **Multi-Class Support:** It naturally handles classification across 26 classes corresponding to the ASL alphabet.

The model takes the 42-dimensional normalized feature vector as input and outputs the predicted alphabet class along with associated probabilities, which are later used for confidence-based filtering during real-time inference.



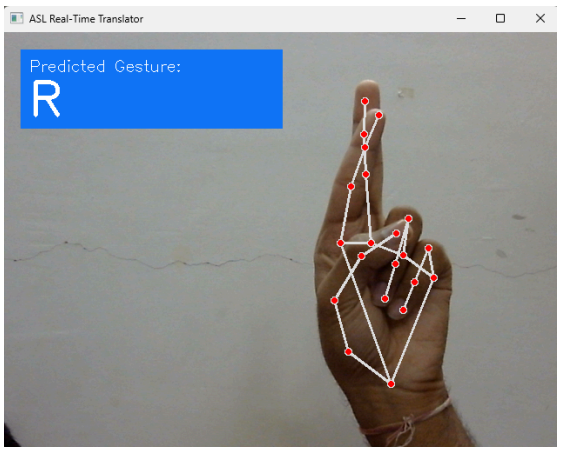
5. Results and Evaluation

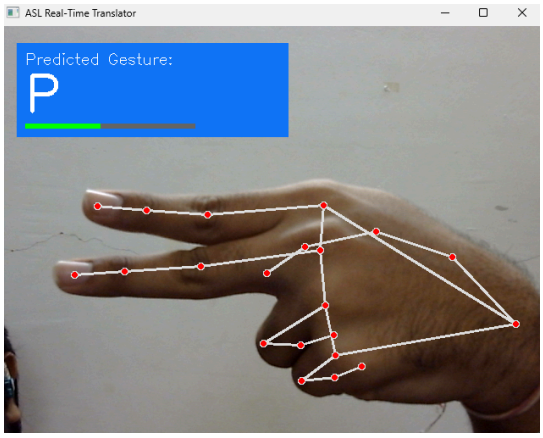
Total Samples: 4681

Training Accuracy: 79.72%

Top Performing Letters: B (94%), C (92%), L (90%)

Challenging Letters: N (62%), A (68%), F (68%)





6. Conclusion

The project demonstrates that an efficient ASL translation system can be built using lightweight techniques. We achieved excellent results through the development of custom normalization and feature engineering methods.

7. Future Work

- Integrate deep learning models

- Add dynamic gesture recognition

- Deploy as mobile/web app